

Oracle ALERT

El paquete **DBMS_ALERT** se utiliza en Oracle Database para proporcionar notificaciones asíncronas basadas en eventos basados en transacciones. Permite que una sesión de base de datos envíe una alerta (una señal) de forma que otras sesiones que estén escuchando activamente actúen inmediatamente al recibirla.

La característica fundamental y más importante de **DBMS_ALERT** es que depende estrictamente del control de transacciones. La alerta se envía efectivamente solo si la transacción que la generó realiza un COMMIT. Si la transacción hace un ROLLBACK, la alerta se descarta de forma automática.

Casos de Uso Principales

El paquete **DBMS_ALERT** se aplica en escenarios donde se requiere una arquitectura dirigida por eventos (Event-Driven) dentro o hacia afuera de la base de datos:

1. Notificación de Cambios en Datos Críticos:

Cuando se inserta o modifica un registro vital (por ejemplo, una transferencia bancaria sospechosa o una orden de alta prioridad), un trigger puede disparar una alerta para que un proceso externo reaccione de inmediato.

2. Sincronización de Cachés Externas:

Aplicaciones de middleware o servicios externos que mantienen datos en memoria pueden "escuchar" alertas de Oracle para purgar o actualizar su caché local únicamente cuando ocurra un cambio real en las tablas.

3. Refresco de Interfaces de Usuario (UI) en Tiempo Real:

Herramientas o paneles de control (dashboards) que necesitan mostrar datos actualizados al segundo sin necesidad de estar haciendo consultas repetitivas (polling) a la base de datos de manera cíclica.

4. Coordinación de Procesos Batch (Lotes):

Indicar a procesos secundarios o daemons en segundo plano que la carga principal de datos ha finalizado correctamente (COMMIT) y que pueden iniciar el procesamiento de la siguiente fase.

Uso Correcto del Paquete (Ejemplo Práctico)

El flujo requiere siempre de dos partes: una sesión **EMISORA** (que publica la alerta) y una o más sesiones **RECEPTORAS** (que esperan la alerta).

Parte 1: Sesión EMISORA (Publicación de la Alerta)

Por lo general, esto se implementa dentro de un trigger de base de datos o un procedimiento almacenado.

```

BEGIN
    --
    -- Tomar en cuenta que sea asincronica esta transaccion
    --
    -- 1. Registrar la señal indicando el nombre de la alerta y un mensaje
opcional
    DBMS_ALERT.SIGNAL(
        name      => 'nuevo_inventario_critico',
        message => 'Stock del artículo XYZ ha caído por debajo del mínimo.'
    );
    --
    -- 2. IMPORTANTE: La alerta NO se enviará hasta que ocurra el COMMIT
    COMMIT;
    --
END;

```

Parte 2: Sesión RECEPTORA (Escucha Activa de la Alerta)

Este bloque se ejecuta en la sesión que necesita reaccionar al evento. Por diseño, el hilo de ejecución se "pausa" de forma eficiente mientras espera el mensaje.

```

DECLARE
    --
    v_message  VARCHAR2(1800);
    --
    /*
        0 significa que se recibió la alerta.
        1 significa que expiró el tiempo de espera
    */
    --
    v_status    INTEGER;
    --
BEGIN
    --
    -- 1. Registrar el interés de esta sesión en la alerta específica
    DBMS_ALERT.REGISTER(
        name => 'nuevo_inventario_critico'
    );
    --
    -- 2. Esperar de forma síncrona a que ocurra la alerta (tiempo de espera:
60 segundos)
    DBMS_ALERT.WAITONE(
        name          => 'nuevo_inventario_critico',
        message       => v_message,
        status        => v_status,
        timeout       => 60
    );

```

```

);
--
-- 3. Evaluar el resultado
IF v_status = 0 THEN
    --
    DBMS_OUTPUT.PUT_LINE('¡Alerta recibida! Mensaje: ' || v_message);
    /*
        TODO: Logica de negocio si se recibe una ALERTA
    */
ELSE
    --
    DBMS_OUTPUT.PUT_LINE('Tiempo de espera agotado sin recibir alertas.');
```

--

```

END IF;
--
-- 4. Desregistrar la alerta al terminar para liberar recursos
DBMS_ALERT.REMOVE(
    name      => 'nuevo_inventario_critico'
);
--
END;
```

Diferencia Clave: DBMS_ALERT vs DBMS_PIPE

Es común confundir estos dos paquetes orientados a la comunicación entre sesiones. Sus diferencias técnicas dictan su uso correcto:

Característica	DBMS_ALERT	DBMS_PIPE
Control de Transacciones	Enlaza al COMMIT. Si hay ROLLBACK no hay mensaje.	Asíncrono inmediato. No le importa el estado de la transacción.
Tipo de Comunicación	One-to-Many (Broadcasting). Todas las sesiones registradas reciben la alerta.	One-to-One (Punto a Punto). El primer lector remueve el mensaje del pipe.
Bloqueo / Espera	Basado en eventos. Eficiente en recursos del sistema operativo.	Basado en memoria compartida (SGA). Muy rápido pero exclusivo.

SEGURIDAD Y EJECUCION

Un usuario con privilegios de administrador (SYS o SYSTEM) debe ejecutar la siguiente sentencia:

```
GRANT EXECUTE ON SYS.DBMS_ALERT TO nombre_usuario;
```

Consideraciones Clave sobre los Permisos

Uso en Bloques Anónimos: Una vez otorgado el permiso anterior, el usuario podrá utilizar DBMS_ALERT directamente en scripts, bloques DECLARE/BEGIN/END o herramientas cliente (como SQL Developer).

Uso dentro de Procedimientos, Funciones o Triggers: Si el usuario necesita crear un objeto almacenado (como un trigger que dispare la alerta al hacer un INSERT), el permiso de EXECUTE debe ser otorgado directamente al usuario (como se muestra en el comando de arriba) y no a través de un ROL (como el rol RESOURCE). **_Si se otorga mediante un rol, el trigger arrojará un error de compilación indicando que el paquete no existe.**

Permisos Adicionales (Acceso a Tablas)

Internamente, DBMS_ALERT gestiona las alertas utilizando tablas del sistema en el esquema SYS (específicamente tablas como SYS.DBMS_ALERT_INFO). Al otorgar el EXECUTE, Oracle maneja internamente estos accesos mediante los privilegios del definidor, por lo que no es necesario otorgar permisos de lectura o escritura en ninguna tabla oculta del sistema.